LAPORAN TUGAS BESAR

PROYEK STRUKTUR DATA DAN ALGORITMA

Disusun Oleh

Kelompok 1 Kelas A1:

1. Dzaky Alfareza Zahari (G1A025005)
2. Aprilizza Auliya (G1A025007)
3. Annisa Rohiimah Sufriyani (G1A025013)
4. Fayola Deeba Fathinah (G1A025028)
5. Melzaqi Al-Faroh (G1A025035)

Nama Asisten Dosen:

1. I Nyoman Dimas Kresna Adryan (G1A023077)
2. Ahsan Nur fauzi (G1A024003)
3. Muhammad Iqbal Zafarullah (G1A024007)
4. M Yusuf Arrayyan (G1A024009)
5. Shintia Bella (G1A024014)
6. Muhammad Kadafi Naufala (G1A024034)
7. Alfarian Adisaputra (G1A024039)
8. Ali Akbar Bermano (G1A024041)
9. Lovien Najla Dhafiyah (G1A024055)
10. Gusti Revalion Putrada (G1A024081)

Dosen Pengampu:

1. Ir. Arie Vatresia, S.T., M.T.I., Ph.D., IPP.
2. Dr. Ulfasari Rafflesia, S.Si., M.Si.

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS BENGKULU

2026

KATA PENGANTAR

Segala puji bagi Allah SWT yang telah memberikan kami kemudahan sehingga kami dapat menyelesaikan laporan ini dengan tepat waktu. Tanpa pertolongan-Nya tentunya kami tidak akan sanggup untuk menyelesaikan laporan ini dengan baik. Shalawat serta salam semoga terlimpah curahkan kepada baginda tercinta kita yaitu Nabi Muhammad SAW yang kita nanti-nantikan syafa'atnya di akhirat nanti.

Penulis mengucapkan syukur kepada Allah SWT atas limpahan nikmat sehat-Nya, baik itu berupa sehat fisik maupun akal pikiran, sehingga penulis mampu untuk menyelesaikan pembuatan laporan tugas besar Proyek Struktur Data dan Algoritma dengan studi kasus Manajemen Inventaris Gudang yang disertai dengan logika Sorting berdasarkan stok paling sedikit dengan menggunakan perangkat lunak bernama VsCode dan Qt Designer.

Penulis tentu menyadari bahwa laporan ini masih jauh dari kata sempurna dan masih banyak terdapat kesalahan serta kekurangan di dalamnya. Untuk itu, penulis mengharapkan kritik serta saran yang membangun dari pembaca untuk laporan ini, supaya laporan ini nantinya dapat menjadi laporan yang lebih baik lagi. Demikian, dan apabila terdapat banyak kesalahan pada laporan ini penulis mohon maaf yang sebesar-besarnya.

Penulis juga mengucapkan terima kasih kepada semua pihak, terutama kepada para dosen pengampu, yaitu Ibu Ir. Arie Vatresia, S.T., M.T.I., Ph.D., IPP. dan Ibu Dr. Ulfasari Rafflesia, S.Si., M.Si. yang telah memberikan kami selaku penulis laporan tugas besar ini kesempatan dan ilmu untuk mengembangkan pengetahuan kami mengenai program berorientasi objek. Lalu, tidak lupa kami juga memberikan ucapan terima kasih kepada para asisten dosen, yakni Muhammad Iqbal Zafarullah dan Muhammad Yusuf Arayyan yang telah membimbing kami selama ini. Kepada teman-teman seperjuangan pun tidak luput dari ucapan terima kasih. Semoga kita semua sukses dalam mempresentasikan hasil tugas besar kita di hari presentasi nanti. Aamiin. Demikian, semoga laporan tugas besar ini dapat bermanfaat. Terima kasih.

Bengkulu, 07 April 2026

Kelompok 1

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
DAFTAR GAMBAR iii
BAB I PENDAHULUAN 1
A. Latar Belakang 1
B. Rumusan Masalah 1
C. Tujuan Penulisan 1
D. Manfaat Penulisan 2
BAB II LANDASAN TEORI 3
A. Konsep Pyton dan Sorting 3
B. Pengenalan Konsep PySide6 3
C. Pengenalan Konsep Qt Designer 4
BAB III SOAL DAN PEMBAHASAN 5
BAB IV PENUTUP 23
A. KESIMPULAN 28
B. SARAN 28
DAFTAR PUSTAKA 29
LAMPIRAN 30

DAFTAR GAMBAR

Gambar 1.1 Design UI Bagian Tentang Kami 5
Gambar 1.2 Design UI Bagian Stok Barang 6
Gambar 1.3 Design UI mencakup bagian Dashboard dan Restock Priority 7
Gambar 1.4 Design UI bagian Dialog Tambah Barang 8
Gambar 1.5 Design UI bagian Dialog Update Stok Barang 9
Gambar 1.6 *Source Code* Implementasi Struktur Data Doubly Linked List. 10
Gambar 1.7 *Source Code* Kontrol UI dan Jendela Dialog 14
Gambar 1.8 *Source Code* Kontrol UI dan Jendela Dialog 14
Gambar 1.9 *Source Code* Kontrol UI dan Jendela Dialog 15
Gambar 1.10 *Source Code* Kontrol UI dan Jendela Dialog 15
Gambar 1.11 *Source code* Kontrol UI dan Jendela Dialog 16
Gambar 1.12 *Source Code* Kontrol UI dan Jendela Dialog 16
Gambar 1.13 Tampilan Output Dashboard 24
Gambar 1.14 Tampilan Output Stok Barang 25
Gambar 1.15 Tampilan Output Restock Priority 26
Gambar 1.16 Tampilan Output Tentang Kami 27

BAB I
PENDAHULUAN

A. Latar Belakang

Dalam era digitalisasi saat ini, efisiensi pengelolaan data menjadi faktor krusial bagi keberhasilan operasional sebuah gudang atau *warehouse*. Pengelolaan inventaris yang dilakukan secara manual seringkali menghadapi kendala, seperti kesalahan pencatatan jumlah stok, kesulitan dalam pengelompokan kategori barang, hingga keterlambatan dalam proses pengisian ulang stok (*restock*).

Masalah utama yang sering muncul adalah kurangnya visibilitas terhadap barang-barang yang jumlahnya sudah menipis. Tanpa sistem yang terorganisir, pengelola gudang harus memeriksa data satu per satu untuk menentukan barang mana yang perlu diprioritaskan untuk dipesan kembali. Hal ini membuang waktu dan berisiko menyebabkan kekosongan stok (*out of stock*) yang menghambat distribusi.

Untuk mengatasi permasalahan tersebut, diperlukan sebuah sistem berbasis komputer yang mampu mengolah data barang (seperti nama, kode SKU, dan kategori) secara efisien. Salah satu penerapan ilmu komputer yang paling relevan dalam kasus ini adalah penggunaan Struktur Data dan Algoritma Pengurutan (*Sorting*). Dengan mengimplementasikan algoritma *sorting* berdasarkan kriteria stok tersedikit, sistem dapat secara otomatis menyajikan data prioritas *restock* secara akurat. Hal ini memungkinkan pengambilan keputusan yang lebih cepat dan tepat dalam manajemen rantai pasok.

B. Rumusan Masalah

1. Bagaimana cara mengimplementasikan struktur data yang tepat untuk menyimpan informasi barang yang meliputi nama, kode SKU, stok, dan kategori?
2. Bagaimana logika algoritma pengurutan (*sorting*) dapat diterapkan untuk menampilkan urutan barang berdasarkan jumlah stok paling sedikit?
3. Sejauh mana penggunaan sistem manajemen inventaris otomatis ini dapat membantu efisiensi proses identifikasi barang yang memerlukan *restock*?

C. Tujuan Penulisan

1. Mengimplementasikan konsep dasar pemrograman ke dalam sebuah aplikasi nyata berupa sistem manajemen inventaris gudang.
2. Memahami dan menerapkan pemrograman berorientasi objek (OOP) dalam merepresentasikan data barang (nama, SKU, stok, dan kategori).
3. Melatih penggunaan struktur data seperti list/array untuk menyimpan dan mengelola data inventaris serta mengasah kemampuan dalam menerapkan algoritma dasar, seperti

sorting (pengurutan) untuk menentukan prioritas restock .

4. Mempelajari pembuatan antarmuka grafis (GUI) menggunakan PySide6 dan Qt Designer.
5. Mengintegrasikan antara desain tampilan (UI) dan logika program dalam satu aplikasi yang utuh.

D. Manfaat Penulisan

1. Bagi Penulis

   Laporan ini bermanfaat sebagai sarana implementasi teori Struktur Data dan Algoritma ke dalam proyek nyata, sekaligus mengasah keterampilan teknis dalam merancang antarmuka GUI menggunakan PySide6 dan Qt Designer serta melatih logika pemecahan masalah secara digital.
2. Bagi Pembaca

   Laporan ini memberikan solusi praktis dalam pemantauan stok secara akurat dan responsif, membantu meningkatkan efisiensi identifikasi barang kritis melalui fitur prioritas *restock*, serta menjadi referensi teknis integrasi desain UI dengan logika Python.
3. Bagi Institusi Pendidikan

   Laporan ini bermanfaat sebagai dokumentasi resmi pencapaian kompetensi mahasiswa dalam penguasaan materi perkuliahan dan menambah koleksi literatur proyek aplikasi yang aplikatif serta fungsional di lingkungan akademis.

BAB II
LANDASAN TEORI

A. Konsep Python dan Sorting

Python merupakan salah satu bahasa pemrograman yang banyak digunakan dalam pengembangan perangkat lunak karena kemudahan sintaks dan fleksibilitasnya. Dalam implementasi algoritma, Python sering digunakan untuk menyelesaikan berbagai permasalahan komputasi, termasuk pengolahan data dan struktur data. Berdasarkan penelitian terbaru, Python tetap menunjukkan kinerja yang cukup baik meskipun merupakan bahasa interpreted, terutama dalam penerapan algoritma seperti topological sorting. Hal ini menunjukkan bahwa Python tidak hanya mudah digunakan, tetapi juga cukup efektif dalam menangani permasalahan algoritmik yang kompleks (Zhang, 2024).

Sorting atau pengurutan data merupakan salah satu konsep dasar dalam ilmu komputer yang memiliki peran penting dalam berbagai aplikasi, seperti pencarian data, pengolahan database, dan optimasi sistem. Proses sorting bertujuan untuk menyusun data dalam urutan tertentu, baik secara menaik maupun menurun, sehingga mempermudah proses analisis dan pengolahan data selanjutnya. Berbagai algoritma sorting seperti Bubble Sort, Selection Sort, Insertion Sort, dan Merge Sort memiliki karakteristik dan tingkat efisiensi yang berbeda, sehingga pemilihan algoritma yang tepat sangat penting dalam meningkatkan performa program (Liu, 2024).

Dalam implementasinya menggunakan Python, algoritma sorting dapat diterapkan dengan lebih sederhana karena struktur sintaks yang jelas dan mudah dipahami. Selain itu, Python juga mendukung berbagai pendekatan dalam pengolahan data sehingga memudahkan pengembang dalam menguji dan membandingkan performa berbagai algoritma sorting. Penelitian menunjukkan bahwa meskipun Python memiliki keterbatasan dalam kecepatan dibandingkan bahasa kompilasi, penggunaannya tetap relevan dalam pembelajaran dan pengembangan sistem karena kemudahan implementasi serta fleksibilitasnya dalam menangani berbagai jenis data.

B. Pengenalan Konsep PySide6

PySide6 merupakan salah satu framework berbasis Qt yang digunakan sebagai binding Python untuk membangun aplikasi Graphical User Interface (GUI). Framework ini memungkinkan pengembang untuk memanfaatkan fitur-fitur Qt dalam lingkungan Python, sehingga mempermudah pembuatan aplikasi desktop yang bersifat interaktif dan lintas platform. PySide6 menjadi solusi yang menggabungkan kemudahan sintaks Python dengan kekuatan framework Qt dalam mengelola komponen antarmuka, event-driven programming, manajemen widget kompleks (Grassucci et al., 2023) menjadikan PySide6 lebih efektif.

Selain itu, PySide6 mendukung konsep event-driven programming melalui mekanisme signal dan slot, yang memungkinkan komunikasi antar komponen aplikasi secara efisien dan terstruktur. PySide6 digunakan dalam pengembangan software analisis film berbasis Python yang menunjukkan bahwa mekanisme signal-slot memungkinkan interaksi antar komponen berlangsung secara fleksibel dan tidak saling bergantung secara langsung (Li et al., 2025). Sistem ini juga mendukung penggunaan multithreading sehingga proses berat seperti analisis data dapat berjalan tanpa mengganggu tampilan antarmuka pengguna, sehingga meningkatkan pengalaman pengguna secara keseluruhan .

Penggunaan PySide6 dalam pengembangan aplikasi juga terbukti mampu meningkatkan efisiensi dan kualitas sistem, terutama pada aplikasi yang membutuhkan interaksi kompleks antara pengguna dan sistem. PySide6 digunakan sebagai dasar dalam membangun antarmuka yang terintegrasi dengan berbagai teknologi seperti pengolahan citra, pengenalan suara, dan kecerdasan buatan. integrasi PySide6 dengan Python memungkinkan pengembangan aplikasi yang tidak hanya interaktif, tetapi juga memiliki performa yang baik serta mudah dikembangkan lebih lanjut . Dengan demikian, PySide6 menjadi salah satu framework penting dalam pengembangan aplikasi GUI modern berbasis Python.

C. Pengenalan Konsep Qt Designer

Qt Designer merupakan alat bantu visual dalam framework Qt yang digunakan untuk merancang antarmuka pengguna (Graphical User Interface/GUI) secara efisien melalui pendekatan drag-and-drop. Qt Designer berperan penting dalam mempercepat proses pengembangan GUI karena memungkinkan perancangan tampilan secara visual tanpa harus menulis kode secara manual (Nabijonovich & Najmiddin, 2024). Integrasi antara Qt Designer dan PyQt memungkinkan pengembang untuk merancang layout antarmuka secara langsung, kemudian menghubungkannya dengan logika program Python. Pendekatan ini mendukung proses rapid prototyping, sehingga pengembang dapat dengan cepat membuat, menguji, dan memperbaiki desain antarmuka aplikasi secara iteratif .

Selain itu, Qt Designer juga mendukung pengembangan aplikasi yang lebih terstruktur melalui penerapan konsep modularitas dan reusable component. penggunaan desain berbasis komponen memungkinkan pengembang untuk membagi antarmuka menjadi bagian-bagian kecil yang dapat digunakan kembali, sehingga meningkatkan efisiensi dan maintainability aplikasi. Dengan adanya fitur visualisasi langsung, pengembang dapat melihat perubahan desain secara real-time dan melakukan penyesuaian dengan tentunya jauh lebih mudah (Zhu, 2023). Selain itu, proses pengembangan menjadi lebih cepat karena tidak perlu menulis kode antarmuka secara manual. Hal ini juga membantu mengurangi kesalahan dalam perancangan tampilan aplikasi.

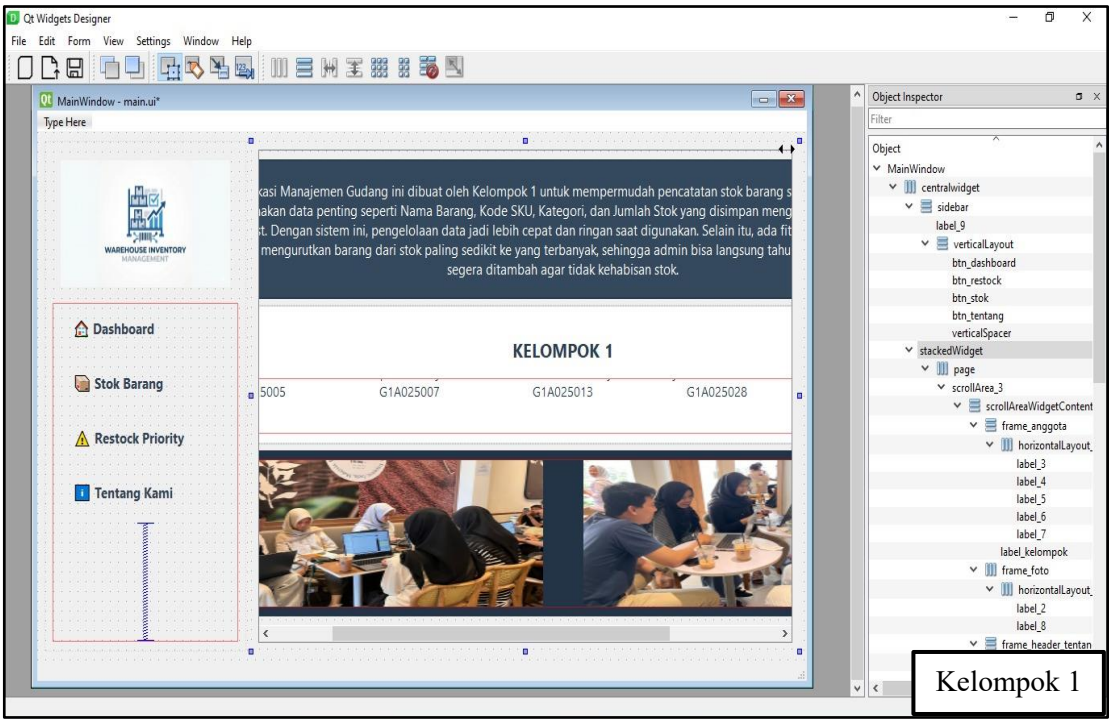
BAB III

SOAL DAN PEMBAHASAN

1. Buatlah sebuah aplikasi Manajemen Inventaris Gudang (Warehouse Management System) berbasis GUI menggunakan PySide6 dengan struktur data Doubly Linked List. Aplikasi harus mampu mengelola data barang dengan atribut Nama Barang, kode SKU, stok, dan kategori

   Jawab:

   PrintScreen *Design UI*:



Gambar 1.1 Design UI Bagian Tentang Kami

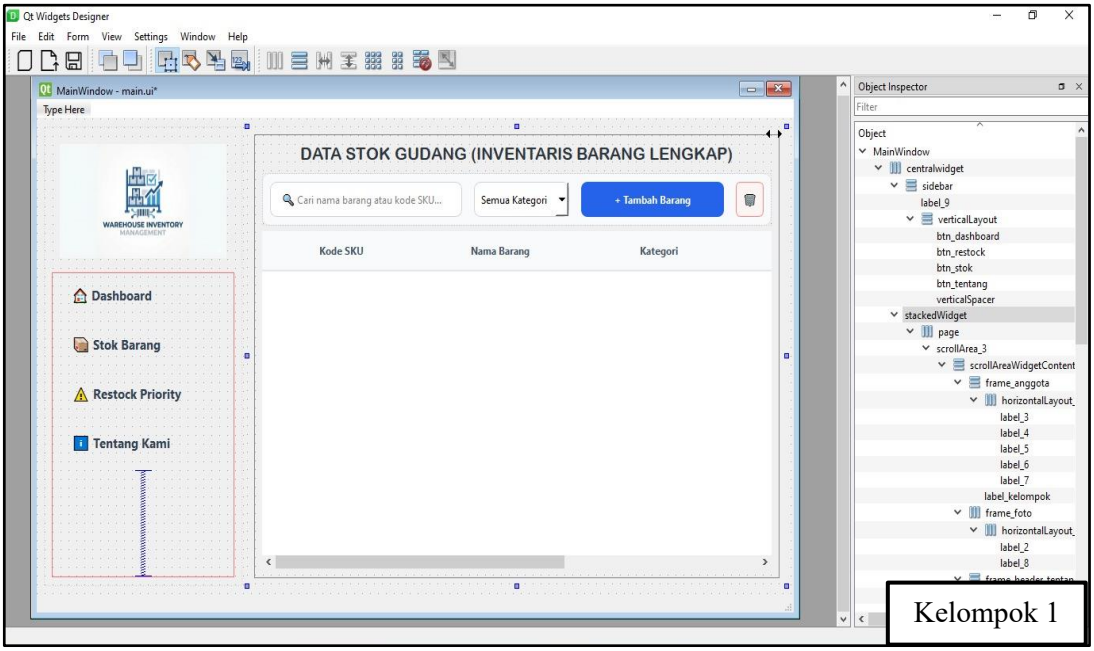
Langkah paling awal dalam pengerjaan ini adalah membangun kerangka utama menggunakan QMainWindow. Dimana kami membagi area menjadi dua fungsional, sisi kiri sebagai Sidebar dan sisi kanan sebagai panel Konten Utama. Sidebar dibangun menggunakan komponen QFrame yang diatur dengan QVBoxLayout untuk menampung logo instansi dan serangkaian tombol navigasi utama. Di area konten utama, dipasang komponen QStackedWidget yang berfungsi sebagai wadah dinamis untuk menampung berbagai halaman. Barulah kami lanjutkan dengan membuat bagian Tentang Kami pada aplikasi ini bertujuan untuk menyajikan informasi identitas pengembang dan deskripsi proyek secara terstruktur.

Dilanjutkan dengan menambahkan halaman baru pada QStackedWidget dan menyematkan QScrollArea sebagai wadah utama. Penggunaan QScrollArea sangat krusial untuk menjamin aksesibilitas konten sehingga dengan komponen ini, seluruh informasi tetap dapat dijangkau melalui mekanisme scrolling meskipun ukuran jendela aplikasi diperkecil

oleh pengguna. Di dalam area tersebut, struktur halaman dibagi menjadi beberapa bagian menggunakan elemen QFrame untuk menciptakan pemisahan visual yang jelas antara area narasi dan area profil. Tahap pertama difokuskan pada penyusunan bagian atas halaman, di mana komponen QLabel berukuran besar diimplementasikan untuk menampilkan judul dan deskripsi umum mengenai tujuan sistem manajemen inventaris. Seluruh elemen pada tahap ini diatur menggunakan QVBoxLayout untuk memastikan tata letak vertikal yang konsisten dan rapi.

Memasuki tahap kedua, pengembang membangun bagian profil melalui frame_anggota. Pada bagian ini, logika tata letak beralih menggunakan QHBoxLayout untuk menyejajarkan identitas anggota kelompok secara horizontal. Tahap ketiga dilanjutkan dengan penyisipan dokumentasi visual pada frame_foto, di mana komponen QLabel dialihfungsikan sebagai penampung gambar dengan memuat file dokumentasi ke dalam properti pixmap. Kemudian dilakukan kustomisasi estetika melalui Style Sheet (CSS Qt) untuk mengatur margin, memberikan efek garis tepi pada bingkai foto, serta menyesuaikan penulisan.

PrintScreen *Design UI*:



Gambar 1.2 Design UI Bagian Stok Barang

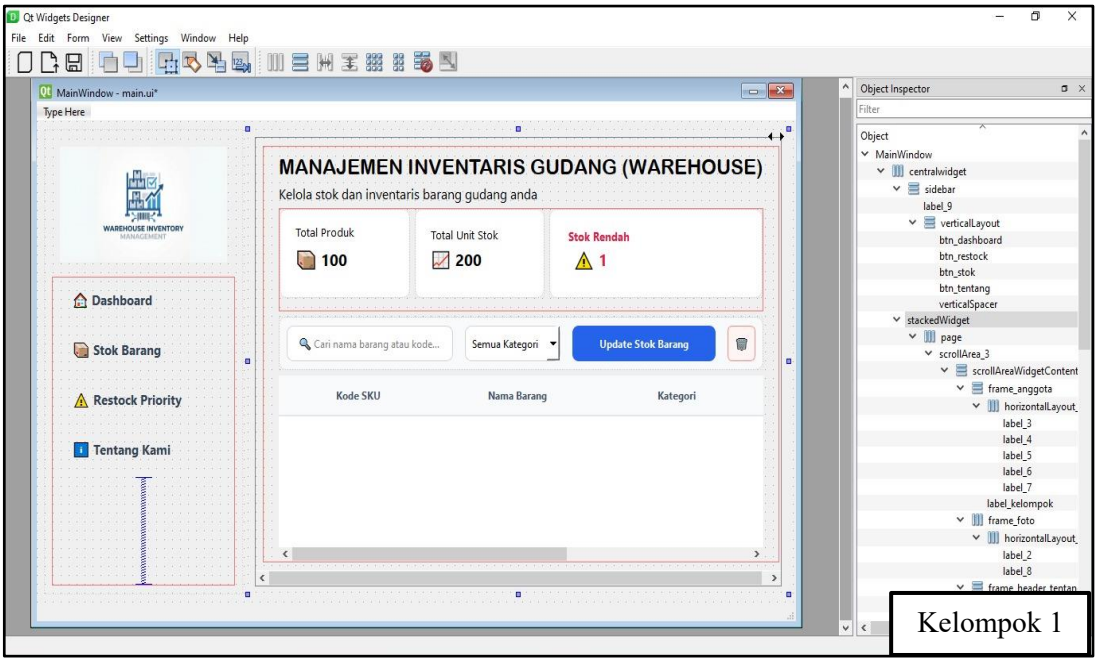
Seperti yang bisa dilihat pada gambar 1.2 pengembangan halaman "Stok Barang" difokuskan pada penyediaan modul manajemen data utama yang memungkinkan pengguna untuk memantau serta mengelola seluruh aset inventaris secara mendetail. Komponen inti yang digunakan dalam halaman ini adalah QTableWidget berukuran besar sebagai elemen penampung data statis maupun dinamis. Tabel ini telah dikonfigurasi dengan kolom-kolom spesifik seperti Kode SKU, Nama Barang, dan Kategori. Untuk mendukung navigasi pada data yang berjumlah banyak, menggunakan QLineEdit sebagai kolom pencarian serta

QComboBox untuk filter kategori, sehingga pengguna dapat menemukan barang dengan lebih efisien.

Proses pengerjaan diawali dengan pembuatan halaman baru di dalam QStackedWidget dan pengaturan properti layout dasar. Tahap pertama adalah penempatan judul halaman menggunakan QLabel dengan penulisan tebal sebagai identitas visual utama. Dilanjutkan dengan menyusun area kontrol di bawah judul menggunakan QHBoxLayout, di mana bar pencarian dan QPushButton ditempatkan dalam satu baris horizontal. Pada bagian ini, terdapat dua tombol aksi utama, “tombol "+ Tambah Barang" yang diberi warna latar biru untuk memicu dialog input, serta tombol hapus (ikon tong sampah) untuk manajemen penghapusan data.

Kemudian pada QTableWidget menentukan jumlah kolom serta mengatur horizontal header agar memenuhi lebar layar secara otomatis. Seluruh elemen ini dibungkus menggunakan QVBoxLayout untuk memastikan hubungan antara judul, baris kontrol, dan tabel tetap konsisten secara vertikal. Sebagai tahap terakhir, dilakukan proses styling melalui Style Sheet untuk menciptakan hierarki Call-to-Action (CTA) yang jelas melalui pemberian warna pada tombol, serta melakukan penamaan objek pada panel Property Editor untuk memudahkan integrasi fungsi CRUD (Create, Read, Update, Delete) pada tahap pemrograman logika.

PrintScreen *Design UI*:



Gambar 1.3 Design UI mencakup bagian Dashboard dan Restock Priority

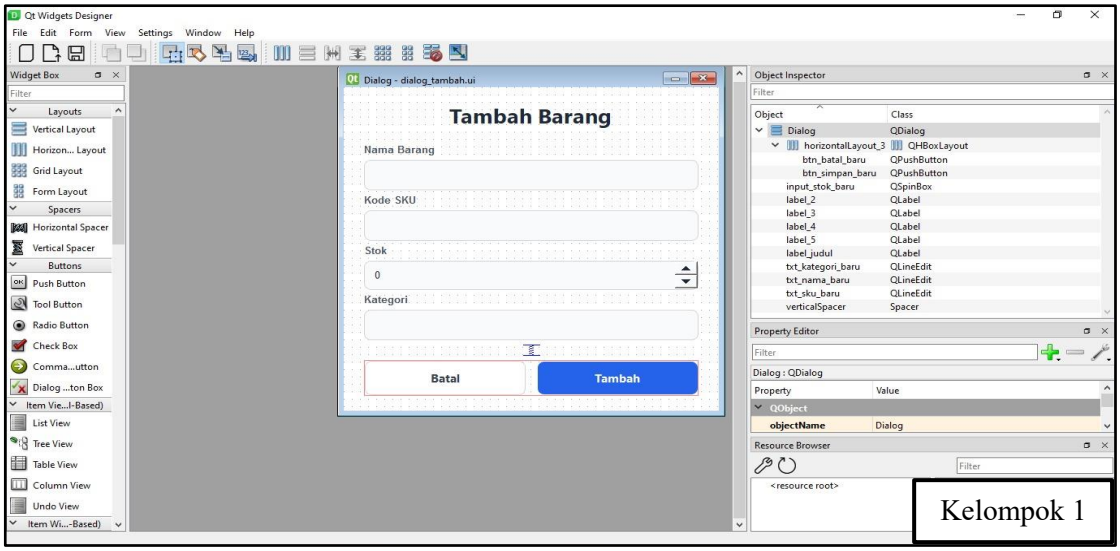
Dari gambar di atas dapat dihilat pengerjaannya kami lanjutkan ke bagian Priority dan Dashboard, yang dirancang sebagai dua komponen yang saling terintegrasi untuk memberikan efisiensi dalam pengambilan keputusan manajemen stok. Dalam alur

pengerjaannya, Restock Priority berperan sebagai "mesin pengolah data kritis" yang menyaring seluruh isi master data barang untuk mengidentifikasi produk mana saja yang jumlah stoknya sudah berada di bawah ambang batas minimum. Halaman ini menggunakan komponen QTableWidget yang secara otomatis menyoroti barang dengan status urgensi tinggi. Pengerjaan bagian ini menjadi dasar logika bagi fungsionalitas Dashboard, di mana data jumlah barang yang memerlukan pengadaan ulang dihitung dan diproses agar bisa ditampilkan secara ringkas.

Sistem QStackedWidget memungkinkan integrasi fungsional di mana halaman Dashboard berperan sebagai antarmuka utama yang merangkum data operasional dari halaman Restock Priority. Sebagai contoh, kartu informasi 'Stok Rendah' pada Dashboard menyajikan akumulasi data secara real-time yang merepresentasikan jumlah baris data atau entitas barang pada daftar prioritas restok. Melalui sinkronisasi logika ini, efisiensi pengguna meningkat karena pemantauan inventaris tidak perlu dilakukan secara parsial, cukup merujuk pada indikator peringatan visual di Dashboard untuk kemudian melakukan navigasi ke halaman Restock Priority guna meninjau rincian barang yang memerlukan pengadaan ulang.

Dilanjutkan melalui sinkronisasi Style Sheet dan penamaan objek yang konsisten. Memastikan bahwa elemen visual pada Dashboard memiliki bahasa desain yang sama dengan halaman Restock Priority, misalnya penggunaan warna merah sebagai penanda peringatan. Dengan struktur yang terbagi namun saling terhubung ini, antara fungsi pemantauan tingkat tinggi (Dashboard) dengan fungsi manajemen detail (Restock Priority),

PrintScreen *Design UI*:



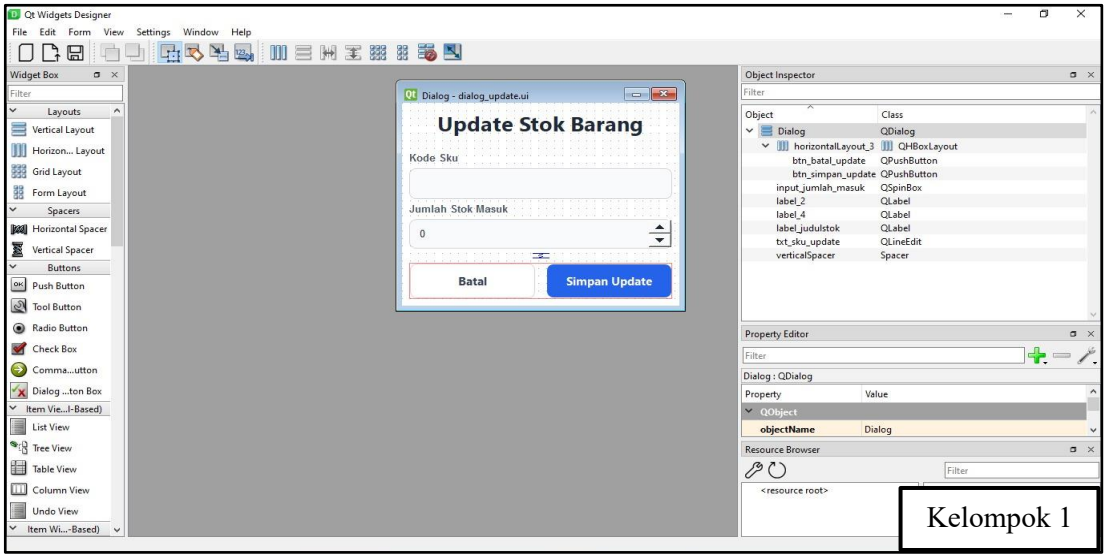
Gambar 1.4 Design UI bagian Dialog Tambah Barang

Dari gambar di atas terdapat rancangan untuk form dialog bernama "Tambah Barang", yang merupakan jendela tambahan bersifat sementara dan tentunya merupakan bagian dari

main window. Dialog “Tambah Barang” ini digunakan untuk menginput data barang baru pada aplikasi Manajemen Inventaris Gudang (Warehouse) yang terhubung dan bisa diakses melalui bagian stock barang dalam desain utama (main window). Dalam form ini menggunakan beberapa jenis widget utama, di antaranya QLabel untuk menampilkan teks statis seperti judul dan label kolom ("Nama Barang", "Kode SKU", dsb.), serta QLineEdit untuk kotak input teks dengan nama objek spesifik Selain itu, terdapat QSpinBox untuk input data angka pada kolom stok, dan dua buah QPushButton untuk aksi eksekusi, yaitu tombol "Batal" dan "Tambah".

Proses pengerjaan dimulai dengan membuat jendela QDialog baru dan melakukan drag-and-drop komponen-komponen yang dibutuhkan dari panel Widget Box ke area kerja. Setelah semua komponen diletakkan, teks bawaan pada label dan tombol diubah sesuai kebutuhan, dan yang sangat penting, nama objek pada Property Editor disesuaikan. Selanjutnya, tata letak antarmuka dirapikan menggunakan fitur Layout. Tombol "Batal" dan "Tambah" dikelompokkan menggunakan QHBoxLayout agar sejajar secara horizontal, dan diletakkan di bawah QSpacerItem vertikal agar posisinya selalu terdorong ke bagian bawah jendela. Terakhir, modifikasi visual diterapkan menggunakan fitur styleSheet (seperti CSS) untuk membesarkan font judul dan memberikan warna latar biru pada tombol "Tambah" agar antarmuka terlihat lebih modern. Hasil akhir dari desain ini nantinya disimpan dalam bentuk file berekstensi .ui yang siap diintegrasikan dengan logika pemrograman Python.

PrintScreen *Design UI*:



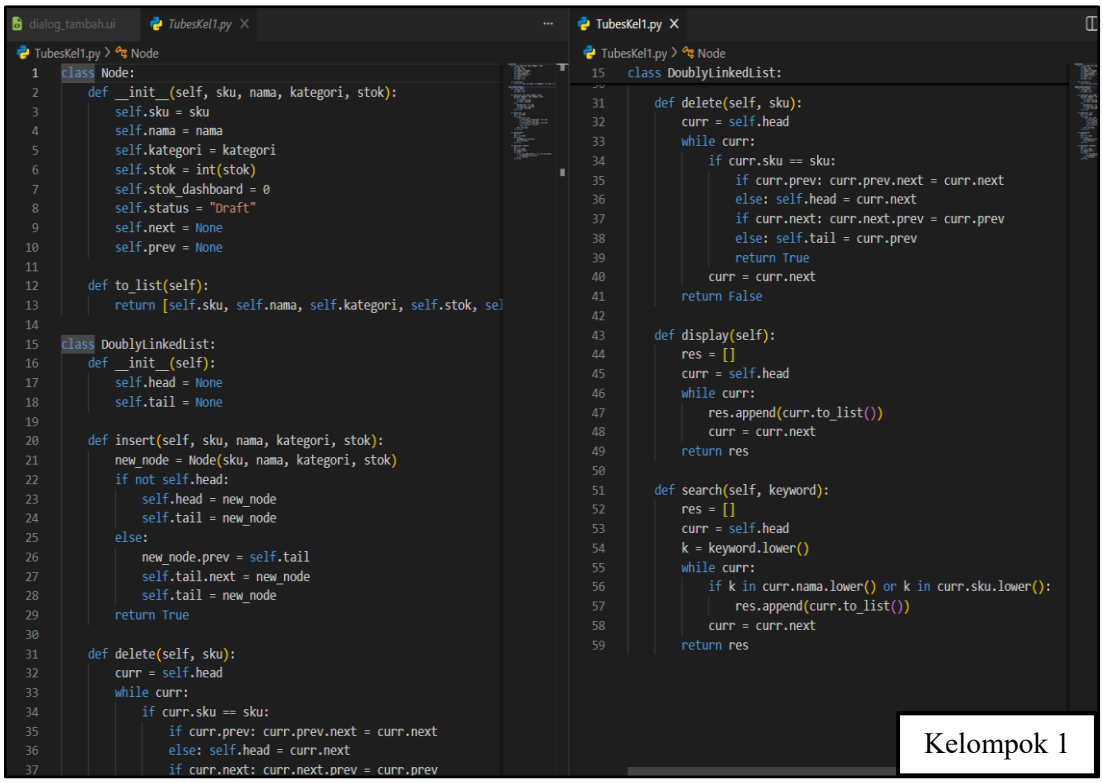
Gambar 1.5 Design UI bagian Dialog Update Stok Barang

Dari gambar di atas terdapat rancangan untuk form dialog bernama "Update Stok Barang" yang merupakan jendela tambahan bersifat sementara dan tentunya merupakan bagian dari main window. Berbeda dengan dialog “Tambah Barang” dialog ini dirancang untuk memperbarui data inventaris, khususnya untuk mencatat penambahan atau perubahan

jumlah stok barang yang sudah ada pada aplikasi Manajemen Inventaris Gudang (Warehouse) yang terhubung dan bisa diakses melalui bagian Dashboard dalam desain utama (main window). Komponen-komponen yang digunakan meliputi QLabel untuk menampilkan teks statis seperti judul dan petunjuk kolom, serta QLineEdit yang digunakan untuk menginput atau menampilkan kode SKU barang. Selain itu, digunakan pula widget QSpinBox untuk memasukkan data angka stok secara presisi, serta dua buah QPushButton sebagai tombol aksi utama, yaitu tombol "Batal" dan "Simpan Update".

Langkah awal dimulai dengan menginisialisasi file baru bermodel Dialog dan melakukan drag-and-drop widget yang diperlukan dari panel Widget Box ke area kerja. Setelah semua widget berada di kanvas, langkah krusial berikutnya adalah penyesuaian atribut pada panel Property Editor, di mana setiap widget diberikan nama objek untuk mempermudah identifikasi saat dihubungkan dengan kode program Python. Penataan posisi dilakukan dengan menerapkan teknik Layouting agar tampilan tetap responsif dan rapi, tombol "Batal" dan "Simpan Update" dikelompokkan dalam sebuah QHBoxLayout agar sejajar secara horizontal, sementara sebuah QSpacerItem disisipkan di atas baris tombol untuk memberikan jarak visual yang seimbang. Tahap akhir melibatkan kustomisasi estetika menggunakan Style Sheet, di mana tombol "Simpan Update" diberikan warna latar biru yang kontras dan teks judul diberikan penebalan serta ukuran yang lebih besar untuk menciptakan hierarki informasi yang jelas sebelum file disimpan dalam format .ui.

PrintScreen *Source Code*:



Gambar 1.6 *Source Code* Implementasi Struktur Data Doubly Linked List.

***Source Code*:**

```
class Node:
    def __init__(self, sku, nama, kategori, stok):
    self.sku = sku
    self.nama = nama
    self.kategori = kategori
    self.stok = int(stok)
      self.stok_dashboard = 0
      self.status = "Draft"
      self.next = None
      self.prev = None
  def to_list(self):
      return [self.sku, self.nama, self.kategori, self.stok, self.status, self.stok_dashboard]
class DoublyLinkedList:
  def __init__(self):
      self.head = None
      self.tail = None
  def insert(self, sku, nama, kategori, stok):
      new_node = Node(sku, nama, kategori, stok)
      if not self.head:
          self.head = new_node
          self.tail = new_node
      else:
          new_node.prev = self.tail
          self.tail.next = new_node
          self.tail = new_node
      return True
  def delete(self, sku):
      curr = self.head
      while curr:
          if curr.sku == sku:
              if curr.prev: curr.prev.next = curr.next
              else: self.head = curr.next
              if curr.next: curr.next.prev = curr.prev
              else: self.tail = curr.prev
```

```
            return True
        curr = curr.next
    return False
def display(self):
    res = []
    curr = self.head
    while curr:
        res.append(curr.to_list())
        curr = curr.next
    return res
def search(self, keyword):
    res = []
    curr = self.head
    k = keyword.lower()
    while curr:

        if k in curr.nama.lower() or k in curr.sku.lower():
            res.append(curr.to_list())
        curr = curr.next
    return res
```

Penjelasan *Source Code*:

Kode program tersebut mengimplementasikan struktur data Doubly Linked List untuk mengelola data inventaris barang secara dinamis dan fleksibel. Struktur ini dibangun menggunakan dua kelas utama, yaitu *Node* dan *DoublyLinkedList*. Kelas *Node* berfungsi sebagai representasi satu unit data atau objek barang yang menyimpan berbagai atribut penting seperti Kode SKU, Nama Barang, Kategori, dan jumlah Stok. Setiap node tidak hanya menyimpan data, tetapi juga memiliki dua penunjuk (pointer), yaitu *next* yang mengarah ke node berikutnya dan *prev* yang mengarah ke node sebelumnya. Keberadaan dua pointer ini menjadi keunggulan utama karena memungkinkan proses traversing (penelusuran) dilakukan secara dua arah, baik dari depan ke belakang maupun sebaliknya. Selain itu, terdapat metode *to_list()* yang berfungsi untuk mengonversi isi node menjadi format list atau array, sehingga data lebih mudah diintegrasikan dengan komponen antarmuka seperti tabel pada GUI atau output berbasis teks.

Sementara itu, kelas *DoublyLinkedList* bertindak sebagai pengelola utama dari keseluruhan kumpulan node yang ada. Kelas ini biasanya memiliki atribut penting seperti

head (penunjuk ke node pertama) dan *tail* (penunjuk ke node terakhir) yang memudahkan pengelolaan data dari kedua ujung struktur. Fungsi *insert* diimplementasikan untuk menambahkan data barang baru ke bagian akhir (tail) dari daftar. Ketika struktur masih kosong, node baru secara otomatis akan menjadi *head* sekaligus *tail*. Namun, jika sudah terdapat data, maka node baru akan dihubungkan dengan node terakhir sebelumnya melalui pengaturan pointer, di mana *next* dari node terakhir lama akan mengarah ke node baru, dan *prev* dari node baru akan mengarah ke node terakhir lama. Proses ini berlangsung dengan kompleksitas waktu yang efisien, yaitu O(1), karena tidak memerlukan pergeseran elemen lain seperti pada struktur array.

Selain fungsi penambahan, kode ini juga menyediakan beberapa fungsi operasional penting lainnya seperti *delete*, *search*, dan *display*. Metode *delete* bekerja dengan mencari node berdasarkan Kode SKU tertentu. Setelah node yang dimaksud ditemukan, sistem akan menyesuaikan pointer dari node sebelumnya dan node sesudahnya untuk "melewati" node yang dihapus, sehingga integritas rantai tetap terjaga. Proses ini juga mempertimbangkan berbagai kondisi khusus, seperti ketika node yang dihapus berada di awal (head), di akhir (tail), atau merupakan satu-satunya elemen dalam daftar.

Fungsi *search* memberikan kemampuan pencarian data yang fleksibel dengan memanfaatkan pencocokan kata kunci (keyword) pada atribut tertentu, seperti nama barang atau Kode SKU. Pencarian ini biasanya dilakukan secara *case-insensitive*, sehingga pengguna tidak perlu memperhatikan perbedaan huruf besar dan kecil. Hal ini meningkatkan kenyamanan dan kemudahan dalam proses pencarian data, terutama ketika jumlah data sudah cukup banyak.

Sementara itu, metode *display* digunakan untuk menampilkan seluruh data yang tersimpan dalam struktur secara berurutan. Proses ini dilakukan dengan menelusuri node mulai dari *head* hingga *tail*, kemudian setiap data node dikonversi ke dalam format list menggunakan metode *to_list()*. Hasil akhirnya dapat berupa kumpulan data yang siap ditampilkan dalam bentuk tabel atau laporan.

Secara keseluruhan, penggunaan struktur Doubly Linked List dalam konteks pengelolaan inventaris ini memberikan berbagai keunggulan, seperti kemudahan dalam penambahan dan penghapusan data tanpa perlu menggeser elemen lain, fleksibilitas dalam navigasi data dua arah, serta efisiensi dalam pengelolaan memori. Struktur ini sangat cocok digunakan untuk sistem yang membutuhkan manipulasi data secara dinamis dan sering mengalami perubahan, seperti aplikasi manajemen stok barang. Dengan implementasi yang tepat, sistem ini dapat membantu menjaga konsistensi data, meningkatkan performa pengolahan, serta memberikan kemudahan dalam pengembangan fitur lanjutan di masa

depan. Selain itu, struktur ini juga memungkinkan pengembangan fitur lanjutan seperti pengurutan (sorting) data berdasarkan kategori atau jumlah stok tanpa perlu mengubah struktur dasar yang sudah ada.

PrintScreen *Source Code*:

dialog_tambah.ui | main.py

main.py > MainWindow > urutkan_priority_dashboard

```python
import sys
from PySide6.QtWidgets import QApplication, QMainWindow, QTableWidgetItem, QDialog, QMessageBox, QHeaderView
from PySide6.QtCore import Qt
from ui_main import Ui_MainWindow
from ui_dialog_tambah import Ui_Dialog as Ui_DialogTambah
from ui_dialog_update import Ui_Dialog as Ui_DialogUpdate
from TubesKel1 import DoublyLinkedList

class DialogTambah(QDialog):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.ui = Ui_DialogTambah()
        self.ui.setupUi(self)
        self.ui.btn_batal_baru.clicked.connect(self.reject)
        self.ui.btn_simpan_baru.clicked.connect(self.accept)

class DialogUpdate(QDialog):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.ui = Ui_DialogUpdate()
        self.ui.setupUi(self)
        self.ui.btn_batal_update.clicked.connect(self.reject)
        self.ui.btn_simpan_update.clicked.connect(self.accept)

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.ui = Ui_MainWindow()
        self.ui.setupUi(self)
        self.inventory = DoublyLinkedList()

        self.siapkan_data_demo()

        self.ui.btn_dashboard.clicked.connect(self.show_dashboard)
        self.ui.btn_stok.clicked.connect(self.show_stok_page)
        self.ui.btn_tentang.clicked.connect(lambda: self.ui.stackedWidget.setCurrentWidge
```

Kelompok 1

Gambar 1.7 *Source Code* Kontrol UI dan Jendela Dialog

main.py > MainWindow > __init__

```python
class MainWindow(QMainWindow):
    def __init__(self):
        kategori_opsi = ["Semua Kategori", "Elektronik", "Aksesoris", "Kabel", "Furnitur", "Jaringan", "ATK", "Makanan", "Minuman"]
        self.ui.combo_kategori_4.clear()
        self.ui.combo_kategori_4.addItems(kategori_opsi)
        self.ui.combo_kategori_4.currentTextChanged.connect(self.filter_kategori_dashboard)

        self.ui.combo_kategori_5.clear()
        self.ui.combo_kategori_5.addItems(kategori_opsi)
        self.ui.combo_kategori_5.currentTextChanged.connect(self.filter_kategori_stok)

        self.ui.input_cari_4.textChanged.connect(self.cari_data)
        self.ui.input_cari_5.textChanged.connect(self.cari_data)

        self.ui.btn_tambah_5.clicked.connect(self.buka_dialog_tambah)
        self.ui.btn_tambah_4.clicked.connect(self.buka_dialog_update_publish)
        self.ui.btn_restock.clicked.connect(self.urutkan_priority_dashboard)

        self.ui.btn_hapus_4.clicked.connect(self.hapus_data)
        self.ui.btn_hapus_5.clicked.connect(self.hapus_data)

        self.ui.tableWidget_4.setSelectionMode(self.ui.tableWidget_4.SelectionMode.ExtendedSelection)
        self.ui.tableWidget_5.setSelectionMode(self.ui.tableWidget_5.SelectionMode.ExtendedSelection)

        self.show_dashboard()

    def siapkan_data_demo(self):
        """Memasukkan 8 data awal ke dalam sistem"""
        data_awal = [
            ("LAP-AZ-001", "Laptop Asus Zenbook", "Elektronik", 5),
            ("MOU-LG-022", "Mouse Logitech G304", "Aksesoris", 45),
            ("MON-SS-010", "Monitor Samsung 24\"", "Elektronik", 8),
            ("KYB-RG-005", "Keyboard Mechanical", "Aksesoris", 12),
            ("CAB-HD-099", "Kabel HDMI 2 Meter", "Kabel", 60),
            ("CHR-SL-003", "Kursi Gaming Secretlab", "Furnitur", 3),
            ("TAB-WD-012", "Meja Kerja Minimalis", "Furnitur", 15),
            ("NET-TP-044", "Router TP-Link AX10", "Jaringan", 7)
```

Kelompok 1

Gambar 1.8 *Source Code* Kontrol UI dan Jendela Dialog

```python
main.py > MainWindow > __init__
class MainWindow(QMainWindow):
    def siapkan_data_demo(self):
        ]
        for sku, nama, kat, stok in data_awal:
            self.inventory.insert(sku, nama, kat, stok)

    def show_dashboard(self):
        self.ui.stackedWidget.setCurrentWidget(self.ui.page_5)
        self.refresh_ui_dashboard()

    def show_stok_page(self):
        self.ui.stackedWidget.setCurrentWidget(self.ui.page_6)
        self.refresh_ui_stok()

    def refresh_ui_dashboard(self, data_list=None):
        if data_list is None:
            all_data = self.inventory.display()
            data_list = [[d[0], d[1], d[2], d[5]] for d in all_data if d[4] == "Published"]

        self.isi_tabel(self.ui.tableWidget_4, data_list)

        total_p = len(data_list)
        total_s = sum(int(i[3]) for i in data_list)
        stok_rendah = len([i for i in data_list if 0 < int(i[3]) < 10])

        self.ui.label_total_produk_4.setText(str(total_p))
        self.ui.label_total_stok_4.setText(str(total_s))
        self.ui.label_stok_rendah_4.setText(str(stok_rendah))

    def refresh_ui_stok(self, data_list=None):
        if data_list is None:
            all_data = self.inventory.display()
            data_list = [[d[0], d[1], d[2], d[3]] for d in all_data]
        self.isi_tabel(self.ui.tableWidget_5, data_list)

    def isi_tabel(self, tabel, data_list):
        tabel.setRowCount(len(data_list))
```

Kelompok 1

Gambar 1.9 *Source Code* Kontrol UI dan Jendela Dialog

```python
main.py > MainWindow > isi_tabel
class MainWindow(QMainWindow):
    def isi_tabel(self, tabel, data_list):
        header = tabel.horizontalHeader()
        tabel.setColumnWidth(0, 40)
        header.setSectionResizeMode(0, QHeaderView.Fixed)
        header.setSectionResizeMode(2, QHeaderView.Stretch)
        header.setSectionResizeMode(4, QHeaderView.ResizeToContents)
        for row, data in enumerate(data_list):
            item_no = QTableWidgetItem(str(row + 1))
            item_no.setTextAlignment(Qt.AlignCenter)
            tabel.setItem(row, 0, item_no)
            for col, val in enumerate(data):
                item = QTableWidgetItem(str(val))
                item.setTextAlignment(Qt.AlignCenter)
                tabel.setItem(row, col + 1, item)

    def filter_kategori_dashboard(self, kat):
        all_data = self.inventory.display()
        pub = [[d[0], d[1], d[2], d[5]] for d in all_data if d[4] == "Published"]
        res = pub if kat == "Semua Kategori" else [d for d in pub if d[2] == kat]
        self.refresh_ui_dashboard(res)

    def filter_kategori_stok(self, kat):
        all_data = self.inventory.display()
        stk = [[d[0], d[1], d[2], d[3]] for d in all_data]
        res = stk if kat == "Semua Kategori" else [d for d in stk if d[2] == kat]
        self.refresh_ui_stok(res)

    def cari_data(self):
        keyword = self.sender().text()
        hasil = self.inventory.search(keyword)
        if self.ui.stackedWidget.currentWidget() == self.ui.page_5:
            res = [[h[0], h[1], h[2], h[5]] for h in hasil if h[4] == "Published"]
            self.refresh_ui_dashboard(res)
        else:
            res = [[h[0], h[1], h[2], h[3]] for h in hasil]
            self.refresh_ui_stok(res)
```

Kelompok 1

Gambar 1.10 *Source Code* Kontrol UI dan Jendela Dialog

```python
main.py > MainWindow > urutkan_priority_dashboard
class MainWindow(QMainWindow):
    def cari_data(self):
        keyword = self.sender().text()
        hasil = self.inventory.search(keyword)
        if self.ui.stackedWidget.currentWidget() == self.ui.page_5:
            res = [[h[0], h[1], h[2], h[5]] for h in hasil if h[4] == "Published"]
            self.refresh_ui_dashboard(res)
        else:
            res = [[h[0], h[1], h[2], h[3]] for h in hasil]
            self.refresh_ui_stok(res)

    def urutkan_priority_dashboard(self):
        """Logika Khusus Kelompok 1: Sort stok paling sedikit & pindah ke Dashboard"""
        all_data = self.inventory.display()
        pub = [[d[0], d[1], d[2], d[5]] for d in all_data if d[4] == "Published"]
        sorted_data = sorted(pub, key=lambda x: int(x[3]))
        self.show_dashboard()
        self.refresh_ui_dashboard(sorted_data)

    def buka_dialog_tambah(self):
        dialog = DialogTambah(self)
        if dialog.exec():
            sku = dialog.ui.txt_sku_baru.text()
            nama = dialog.ui.txt_nama_baru.text()
            kat = dialog.ui.txt_kategori_baru.text()
            stok = dialog.ui.input_stok_baru.value()
            if sku and nama:
                self.inventory.insert(sku, nama, kat, int(stok))
                self.refresh_ui_stok()

    def buka_dialog_update_publish(self):
        dialog = DialogUpdate(self)
        if dialog.exec():
            sku = dialog.ui.txt_sku_update.text()
            jml = dialog.ui.input_jumlah_masuk.value()
            curr = self.inventory.head
            while curr:
```

Kelompok 1

Gambar 1.11 *Source code* Kontrol UI dan Jendela Dialog

```python
dialog_tambah.ui   main.py X
main.py > MainWindow > buka_dialog_update_publish
class MainWindow(QMainWindow):
    def buka_dialog_update_publish(self):
            while curr:
                if curr.sku == sku:
                    if curr.stok >= jml:
                        curr.stok -= jml
                        curr.stok_dashboard += jml
                        curr.status = "Published"
                        self.refresh_ui_dashboard()
                        self.refresh_ui_stok()
                        return
                    else:
                        QMessageBox.warning(self, "Gagal", "Stok gudang tidak cukup!")
                        return
                curr = curr.next
            QMessageBox.warning(self, "Gagal", "SKU tidak ditemukan!")
    def hapus_data(self):
        is_dash = self.ui.stackedWidget.currentWidget() == self.ui.page_5
        tabel = self.ui.tableWidget_4 if is_dash else self.ui.tableWidget_5
        indices = tabel.selectionModel().selectedRows()
        if not indices: return
        rows = sorted([i.row() for i in indices], reverse=True)
        if is_dash:
            if QMessageBox.question(self, "Konfirmasi", "Tarik kembali ke Gudang?") == QMessageBox.
                for r in rows:
                    sku = tabel.item(r, 1).text()
                    curr = self.inventory.head
                    while curr:
                        if curr.sku == sku:
                            curr.stok += curr.stok_dashboard
                            curr.stok_dashboard = 0
                            curr.status = "Draft"
                            break
                        curr = curr.next
        else:
            if QMessageBox.question(self, "Konfirmasi", "Hapus data permanen?") == QMessageBox.Yes:
                for r in rows:
```

```python
main.py X
main.py > MainWindow > buka_dialog_update_pu
class MainWindow(QMainWindow):
    def hapus_data(self):
        self.refresh_ui_dashboar
        self.refresh_ui_stok()

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())
```

Kelompok 1

Gambar 1.12 *Source Code* Kontrol UI dan Jendela Dialog

***Source Code*:**

```
import sys
from PySide6.QtWidgets import QApplication, QMainWindow, QTableWidgetItem, QDialog, QMessageBox, QHeaderView
from PySide6.QtCore import Qt
from ui_main import Ui_MainWindow
from ui_dialog_tambah import Ui_Dialog as Ui_DialogTambah
from ui_dialog_update import Ui_Dialog as Ui_DialogUpdate
from TubesKel1 import DoublyLinkedList

class DialogTambah(QDialog):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.ui = Ui_DialogTambah()
        self.ui.setupUi(self)
        self.ui.btn_batal_baru.clicked.connect(self.reject)
        self.ui.btn_simpan_baru.clicked.connect(self.accept)

class DialogUpdate(QDialog):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.ui = Ui_DialogUpdate()
        self.ui.setupUi(self)
        self.ui.btn_batal_update.clicked.connect(self.reject)
        self.ui.btn_simpan_update.clicked.connect(self.accept)

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.ui = Ui_MainWindow()
        self.ui.setupUi(self)
        self.inventory = DoublyLinkedList()

        self.siapkan_data_demo()
```

```
        self.ui.btn_dashboard.clicked.connect(self.show_dashboard)
        self.ui.btn_stok.clicked.connect(self.show_stok_page)
        self.ui.btn_tentang.clicked.connect(lambda:
        self.ui.stackedWidget.setCurrentWidget(self.ui.page))

        kategori_opsi = ["Semua Kategori", "Elektronik", "Aksesoris", "Kabel", "Furnitur",
"Jaringan", "ATK", "Makanan", "Minuman"]
        self.ui.combo_kategori_4.clear()
        self.ui.combo_kategori_4.addItems(kategori_opsi)
        self.ui.combo_kategori_4.currentTextChanged.connect(self.filter_kategori_dashboar
d)

        self.ui.combo_kategori_5.clear()
        self.ui.combo_kategori_5.addItems(kategori_opsi)
        self.ui.combo_kategori_5.currentTextChanged.connect(self.filter_kategori_stok)

        self.ui.input_cari_4.textChanged.connect(self.cari_data)
        self.ui.input_cari_5.textChanged.connect(self.cari_data)

        self.ui.btn_tambah_5.clicked.connect(self.buka_dialog_tambah)
        self.ui.btn_tambah_4.clicked.connect(self.buka_dialog_update_publish)
        self.ui.btn_restock.clicked.connect(self.urutkan_priority_dashboard)

        self.ui.btn_hapus_4.clicked.connect(self.hapus_data)
        self.ui.btn_hapus_5.clicked.connect(self.hapus_data)

        self.ui.tableWidget_4.setSelectionMode(self.ui.tableWidget_4.SelectionMode.Exten
dedSelection)
        self.ui.tableWidget_5.setSelectionMode(self.ui.tableWidget_5.SelectionMode.Exten
dedSelection)

        self.show_dashboard()

    def siapkan_data_demo(self):
        """Memasukkan 8 data awal ke dalam sistem"""
```

```
        data_awal = [
            ("LAP-AZ-001", "Laptop Asus Zenbook", "Elektronik", 5),
            ("MOU-LG-022", "Mouse Logitech G304", "Aksesoris", 45),
            ("MON-SS-010", "Monitor Samsung 24\"", "Elektronik", 8),
            ("KYB-RG-005", "Keyboard Mechanical", "Aksesoris", 12),
            ("CAB-HD-099", "Kabel HDMI 2 Meter", "Kabel", 60),
            ("CHR-SL-003", "Kursi Gaming Secretlab", "Furnitur", 3),
            ("TAB-WD-012", "Meja Kerja Minimalis", "Furnitur", 15),
            ("NET-TP-044", "Router TP-Link AX10", "Jaringan", 7)
        ]
        for sku, nama, kat, stok in data_awal:
            self.inventory.insert(sku, nama, kat, stok)

    def show_dashboard(self):
        self.ui.stackedWidget.setCurrentWidget(self.ui.page_5)
        self.refresh_ui_dashboard()

    def show_stok_page(self):
        self.ui.stackedWidget.setCurrentWidget(self.ui.page_6)
        self.refresh_ui_stok()

    def refresh_ui_dashboard(self, data_list=None):
        if data_list is None:
            all_data = self.inventory.display()
            data_list = [[d[0], d[1], d[2], d[5]] for d in all_data if d[4] == "Published"]

        self.isi_tabel(self.ui.tableWidget_4, data_list)

        total_p = len(data_list)
        total_s = sum(int(i[3]) for i in data_list)
        stok_rendah = len([i for i in data_list if 0 < int(i[3]) < 10])

        self.ui.label_total_produk_4.setText(str(total_p))
        self.ui.label_total_stok_4.setText(str(total_s))
        self.ui.label_stok_rendah_4.setText(str(stok_rendah))
```

```
def refresh_ui_stok(self, data_list=None):
    if data_list is None:
        all_data = self.inventory.display()
        data_list = [[d[0], d[1], d[2], d[3]] for d in all_data]
    self.isi_tabel(self.ui.tableWidget_5, data_list)

def isi_tabel(self, tabel, data_list):
    tabel.setRowCount(len(data_list))

    header = tabel.horizontalHeader()
    tabel.setColumnWidth(0, 40)
    header.setSectionResizeMode(0, QHeaderView.Fixed)
    header.setSectionResizeMode(2, QHeaderView.Stretch)
    header.setSectionResizeMode(4, QHeaderView.ResizeToContents)
    for row, data in enumerate(data_list):
        item_no = QTableWidgetItem(str(row + 1))
        item_no.setTextAlignment(Qt.AlignCenter)
        tabel.setItem(row, 0, item_no)
        for col, val in enumerate(data):
            item = QTableWidgetItem(str(val))
            item.setTextAlignment(Qt.AlignCenter)
            tabel.setItem(row, col + 1, item)

def filter_kategori_dashboard(self, kat):
    all_data = self.inventory.display()
    pub = [[d[0], d[1], d[2], d[5]] for d in all_data if d[4] == "Published"]
    res = pub if kat == "Semua Kategori" else [d for d in pub if d[2] == kat]
    self.refresh_ui_dashboard(res)

def filter_kategori_stok(self, kat):
    all_data = self.inventory.display()
    stk = [[d[0], d[1], d[2], d[3]] for d in all_data]
    res = stk if kat == "Semua Kategori" else [d for d in stk if d[2] == kat]
    self.refresh_ui_stok(res)
```

```
def cari_data(self):
    keyword = self.sender().text()
    hasil = self.inventory.search(keyword)
    if self.ui.stackedWidget.currentWidget() == self.ui.page_5:
        res = [[h[0], h[1], h[2], h[5]] for h in hasil if h[4] == "Published"]
        self.refresh_ui_dashboard(res)
    else:
        res = [[h[0], h[1], h[2], h[3]] for h in hasil]
        self.refresh_ui_stok(res)

def urutkan_priority_dashboard(self):
    """Logika Khusus Kelompok 1: Sort stok paling sedikit & pindah ke Dashboard"""
    all_data = self.inventory.display()
    pub = [[d[0], d[1], d[2], d[5]] for d in all_data if d[4] == "Published"]
    sorted_data = sorted(pub, key=lambda x: int(x[3]))
    self.show_dashboard()
    self.refresh_ui_dashboard(sorted_data)

def buka_dialog_tambah(self):
    dialog = DialogTambah(self)
    if dialog.exec():
        sku = dialog.ui.txt_sku_baru.text()
        nama = dialog.ui.txt_nama_baru.text()
        kat = dialog.ui.txt_kategori_baru.text()
        stok = dialog.ui.input_stok_baru.value()
        if sku and nama:
            self.inventory.insert(sku, nama, kat, int(stok))
            self.refresh_ui_stok()

def buka_dialog_update_publish(self):
    dialog = DialogUpdate(self)
    if dialog.exec():
        sku = dialog.ui.txt_sku_update.text()
        jml = dialog.ui.input_jumlah_masuk.value()
```

```
            curr = self.inventory.head
            while curr:
                if curr.sku == sku:
                    if curr.stok >= jml:
                        curr.stok -= jml
                        curr.stok_dashboard += jml
                        curr.status = "Published"
                        self.refresh_ui_dashboard()
                        self.refresh_ui_stok()
                        return
                    else:
                        QMessageBox.warning(self, "Gagal", "Stok gudang tidak cukup!")
                        return
                curr = curr.next
            QMessageBox.warning(self, "Gagal", "SKU tidak ditemukan!")
    def hapus_data(self):
        is_dash = self.ui.stackedWidget.currentWidget() == self.ui.page_5
        tabel = self.ui.tableWidget_4 if is_dash else self.ui.tableWidget_5
        indices = tabel.selectionModel().selectedRows()
        if not indices: return
        rows = sorted([i.row() for i in indices], reverse=True)
        if is_dash:
            if QMessageBox.question(self, "Konfirmasi", "Tarik kembali ke Gudang?") ==
QMessageBox.Yes:
                for r in rows:
                    sku = tabel.item(r, 1).text()
                    curr = self.inventory.head
                    while curr:
                        if curr.sku == sku:
                            curr.stok += curr.stok_dashboard
                            curr.stok_dashboard = 0
                            curr.status = "Draft"
                            break
                        curr = curr.next
        else:
```

```
            if QMessageBox.question(self, "Konfirmasi", "Hapus data permanen?") == QMessageBox.Yes:
                for r in rows:
                    self.inventory.delete(tabel.item(r, 1).text())
        self.refresh_ui_dashboard()
        self.refresh_ui_stok()
if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())
```

Penjelasan *Source Code*:

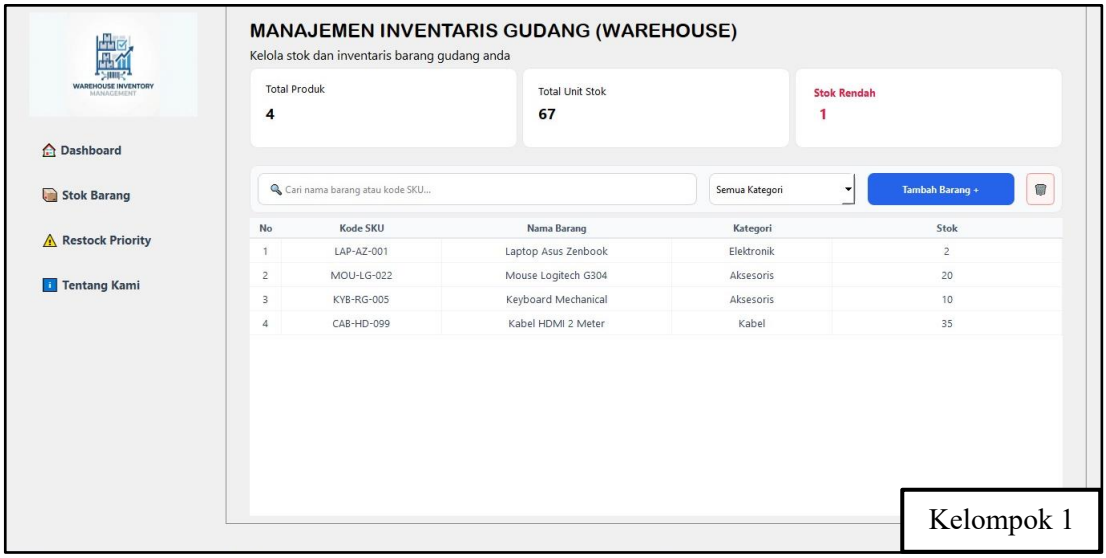
Source code ini merupakan implementasi aplikasi desktop berbasis GUI menggunakan library PySide6 pada bahasa pemrograman Python. Program ini dirancang untuk mengelola data inventaris barang dengan memanfaatkan struktur data Doubly Linked List yang diimpor dari modul TubesKel1. Selain itu, file antarmuka (UI) yang digunakan berasal dari hasil desain menggunakan Qt Designer, yaitu ui_main, ui_dialog_tambah, dan ui_dialog_update. Penggunaan PySide6 memungkinkan pembuatan antarmuka grafis yang interaktif, sedangkan struktur data digunakan untuk mengelola data secara dinamis.

Pada bagian awal kode, dilakukan proses impor berbagai modul yang dibutuhkan seperti QApplication, QMainWindow, QDialog, dan komponen lainnya dari PySide6. Selain itu, diimpor juga Qt dari QtCore untuk pengaturan alignment serta QHeaderView untuk membantu mengatur tampilan tabel. Modul UI yang dihasilkan dari Qt Designer kemudian dihubungkan dengan kode Python agar tampilan yang telah didesain dapat berfungsi secara interaktif. Struktur data DoublyLinkedList digunakan sebagai media penyimpanan utama data inventaris.

Kelas DialogTambah dan DialogUpdate merupakan turunan dari QDialog yang berfungsi sebagai jendela pop-up untuk menambahkan dan memperbarui data. Pada masing-masing kelas dilakukan inisialisasi UI serta penghubungan tombol dengan fungsi tertentu menggunakan metode clicked.connect(). Tombol batal akan menjalankan fungsi reject() untuk menutup dialog tanpa menyimpan data, sedangkan tombol simpan akan menjalankan fungsi accept() untuk memproses input yang diberikan pengguna.

Dengan adanya integrasi antara struktur data dan antarmuka grafis, memungkinkan pengguna untuk melakukan pengelolaan stok secara lebih cepat, akurat, dan terorganisir dalam satu sistem terpadu.

PrintScreen Output:



Gambar 1.13 Tampilan Output Dashboard

Penjelasan Output:

Tampilan Dashboard pada aplikasi ini merupakan halaman utama yang menampilkan ringkasan data inventaris gudang secara keseluruhan. Pada bagian atas, terdapat judul "Manajemen Inventaris Gudang (Warehouse)" yang menunjukkan fungsi utama aplikasi, disertai deskripsi singkat untuk membantu pengguna memahami tujuan sistem. Di bawahnya, terdapat tiga kartu informasi (summary card) yaitu Total Produk, Total Unit Stok, dan Stok Rendah. Nilai yang ditampilkan (misalnya 4 produk, 67 unit stok, dan 1 stok rendah) merupakan hasil perhitungan langsung dari data yang tersimpan dalam struktur *Doubly Linked List*.

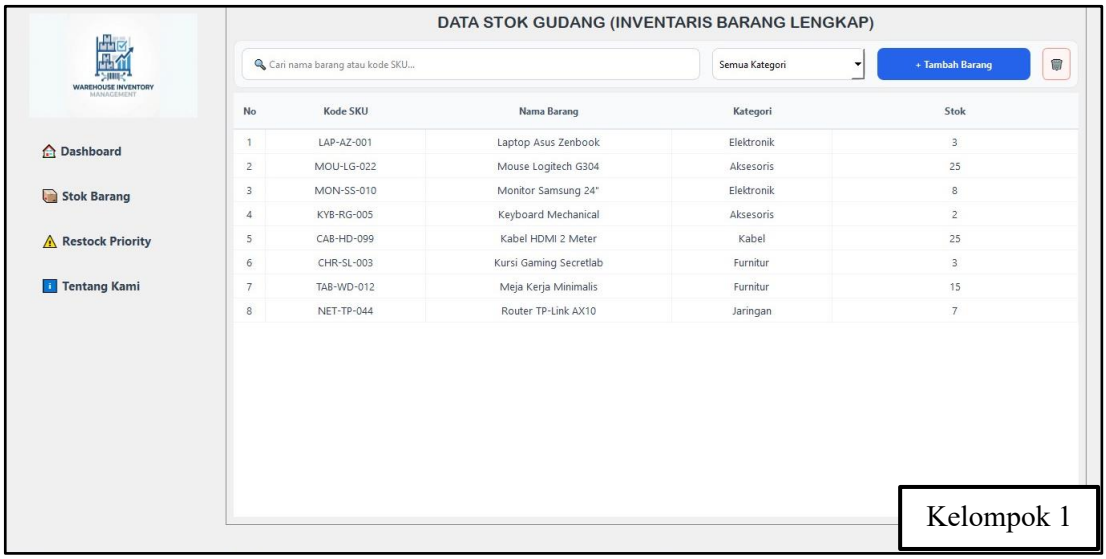
Pada bagian tengah, terdapat fitur pencarian yang memungkinkan pengguna mencari barang berdasarkan nama atau kode SKU, serta dropdown filter kategori yang saat ini masih berada pada kondisi default yaitu "Semua Kategori". Karena data belum disorting, maka seluruh data yang ditampilkan masih mengikuti urutan awal saat dimasukkan (berdasarkan proses insert), bukan berdasarkan prioritas tertentu seperti stok terendah atau kategori tertentu. Tombol "Tambah Barang +" juga tersedia untuk menambahkan data baru ke dalam sistem.

Bagian utama dashboard menampilkan tabel data inventaris yang berisi beberapa kolom seperti No, Kode SKU, Nama Barang, Kategori, dan Stok. Data ditampilkan secara berurutan dari atas ke bawah sesuai urutan input, sehingga belum ada pengelompokan atau pengurutan khusus. Misalnya, barang dengan stok kecil belum tentu berada di posisi atas karena fitur sorting belum diterapkan pada tampilan ini.

Secara keseluruhan, output dashboard ini berfungsi sebagai tampilan awal untuk memonitor kondisi inventaris secara umum sebelum dilakukan proses filter atau sorting

lebih lanjut. Tampilan ini juga menunjukkan bahwa sistem sudah berhasil mengintegrasikan data dari struktur *Doubly Linked List* ke dalam antarmuka GUI yang dibuat menggunakan Qt Designer, sehingga data dapat ditampilkan secara dinamis dan interaktif.

Printscreen Output:



Gambar 1.14 Tampilan Output Stok Barang

Penjelasan Output:

Menu Stok Barang merupakan halaman yang menampilkan seluruh data inventaris secara lengkap dan lebih detail dibandingkan dashboard. Pada bagian atas terdapat judul "Data Stok Gudang (Inventaris Barang Lengkap)" yang menunjukkan bahwa halaman ini berfungsi untuk melihat keseluruhan data barang yang tersimpan dalam sistem.

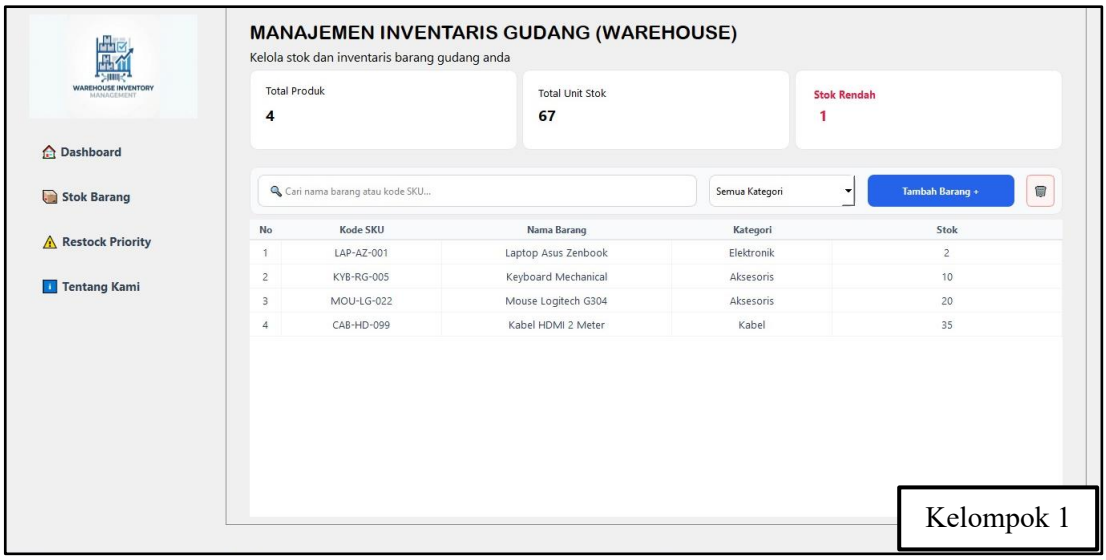
Di bawah judul, tersedia fitur pencarian yang memungkinkan pengguna mencari barang berdasarkan nama atau kode SKU. Selain itu, terdapat dropdown "Semua Kategori" yang digunakan untuk memfilter data berdasarkan kategori tertentu seperti Elektronik, Aksesoris, Furnitur, dan lainnya. Terdapat juga tombol "+ Tambah Barang" yang berfungsi untuk menambahkan data inventaris baru, serta tombol hapus (ikon tempat sampah) untuk menghapus data yang dipilih.

Bagian utama halaman ini adalah tabel data yang menampilkan informasi inventaris secara terstruktur. Tabel terdiri dari beberapa kolom, yaitu No, Kode SKU, Nama Barang, Kategori, dan Stok. Setiap baris merepresentasikan satu data barang yang tersimpan dalam struktur *Doubly Linked List*. Data ditampilkan secara berurutan sesuai dengan urutan input (insert), sehingga belum melalui proses pengurutan tertentu.

Dari tampilan tersebut, pengguna dapat melihat jumlah stok masing-masing barang secara langsung, seperti Laptop Asus Zenbook, Mouse Logitech G304, hingga Router TP-Link AX10 dengan kategori yang berbeda-beda. Hal ini memudahkan pengguna dalam memantau ketersediaan barang di gudang secara menyeluruh.

Secara keseluruhan, menu Stok Barang berfungsi sebagai pusat data inventaris yang memungkinkan pengguna untuk melihat, mencari, menambah, dan menghapus data dengan mudah. Integrasi antara struktur data dan antarmuka GUI membuat data dapat ditampilkan secara dinamis, sehingga setiap perubahan pada data akan langsung tercermin pada tabel yang ditampilkan.

Printscreen Output:



Gambar 1.15 Tampilan Output Restock Priority

Penjelasan Output:

Menu Restock Priority merupakan halaman yang digunakan untuk menampilkan data inventaris berdasarkan prioritas pengisian ulang stok (restock). Pada tampilan ini, data barang telah melalui proses sorting berdasarkan jumlah stok dari yang paling sedikit ke yang paling banyak. Hal ini bertujuan agar pengguna dapat dengan mudah mengidentifikasi barang mana yang harus segera dilakukan restock.

Pada bagian atas, tampilan masih menampilkan ringkasan seperti Total Produk, Total Unit Stok, dan Stok Rendah yang memberikan gambaran umum kondisi inventaris. Namun, perbedaan utama pada menu ini terletak pada urutan data di dalam tabel yang sudah tidak lagi berdasarkan urutan input, melainkan berdasarkan prioritas stok terendah.

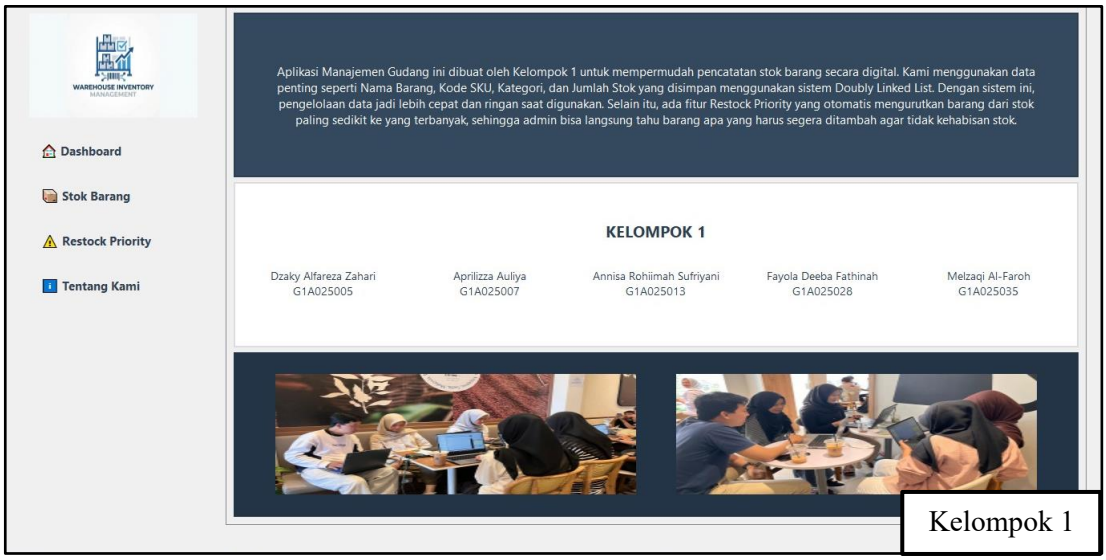
Tabel yang ditampilkan tetap terdiri dari kolom No, Kode SKU, Nama Barang, Kategori, dan Stok, tetapi urutannya telah disusun dari stok terkecil ke terbesar. Sebagai contoh, barang dengan stok paling sedikit seperti *Laptop Asus Zenbook* dengan stok 2 akan berada di posisi teratas, diikuti oleh barang lain dengan stok yang lebih besar seperti keyboard, mouse, hingga kabel HDMI. Dengan urutan ini, pengguna dapat langsung fokus pada barang yang membutuhkan perhatian lebih.

Fitur pencarian dan filter kategori juga tetap tersedia untuk membantu pengguna menyaring data sesuai kebutuhan, meskipun dalam kondisi sudah disorting. Hal ini

memberikan fleksibilitas tambahan, misalnya untuk melihat prioritas restock hanya pada kategori tertentu.

Secara keseluruhan, menu Restock Priority berfungsi sebagai alat bantu pengambilan keputusan dalam manajemen inventaris. Dengan menampilkan data yang telah diurutkan berdasarkan stok terendah, sistem membantu pengguna untuk lebih cepat menentukan barang yang perlu segera ditambah, sehingga mengurangi risiko kehabisan stok dan meningkatkan efisiensi pengelolaan gudang.

Printscreen Output:



Gambar 1.16 Tampilan Output Tentang Kami

Penjelasan Output:

Menu Tentang Kami merupakan halaman yang berisi informasi mengenai latar belakang pembuatan aplikasi serta identitas tim pengembang. Pada bagian atas ditampilkan deskripsi singkat yang menjelaskan bahwa aplikasi manajemen gudang ini dibuat untuk mempermudah pencatatan dan pengelolaan stok barang secara digital. Ini menggunakan struktur data *Doubly Linked List*, serta fitur unggulan seperti Restock Priority yang membantu pengguna dalam menentukan prioritas pengisian ulang stok.

Pada bagian tengah, terdapat informasi mengenai Kelompok 1 sebagai tim pengembang aplikasi. Di bagian ini ditampilkan nama-nama anggota beserta nomor identitasnya, sehingga memberikan kejelasan mengenai siapa saja yang terlibat dalam pembuatan sistem. Penyusunan informasi ini dibuat rapi dan terpusat agar mudah dibaca dan dipahami oleh pengguna.

Di bagian bawah, ditampilkan dokumentasi berupa gambar kegiatan tim selama proses pengerjaan proyek. Gambar tersebut berfungsi sebagai bukti visual kolaborasi tim dalam mengembangkan aplikasi, sekaligus memberikan kesan lebih informatif dan menarik pada tampilan.

BAB IV
PENUTUP

A. KESIMPULAN

Proyek Manajemen Inventaris Gudang (Warehouse) ini merupakan implementasi yang cukup komprehensif dengan memanfaatkan teknologi Python, PySide6, dan Qt Designer sebagai dasar pengembangannya. Dari sisi bahasa pemrograman, Python digunakan karena memiliki sintaks yang sederhana namun мощный dalam pengolahan data, sehingga sangat cocok untuk membangun logika sistem inventaris. Konsep utama yang diterapkan tetap berfokus pada pemrograman berorientasi objek (OOP), di mana setiap barang direpresentasikan sebagai objek dengan atribut seperti nama, kode SKU, stok, dan kategori. Pendekatan ini membuat sistem lebih terstruktur, mudah dikembangkan, dan scalable untuk kebutuhan yang lebih kompleks.

Dalam pengelolaan data, proyek ini menggunakan struktur data seperti list atau array untuk menyimpan kumpulan objek barang. Selanjutnya, diterapkan algoritma sorting (pengurutan) untuk mengurutkan data berdasarkan jumlah stok paling sedikit, yang berfungsi sebagai dasar dalam menentukan prioritas restock. Logika ini diperkuat dengan penggunaan percabangan (if-else) untuk mengidentifikasi kondisi stok tertentu. Dengan demikian, sistem tidak hanya menyimpan data, tetapi juga mampu melakukan analisis sederhana terhadap data tersebut.

Keunggulan utama dari proyek ini terletak pada penggunaan PySide6 yang memungkinkan pembuatan Graphical User Interface (GUI) berbasis framework Qt, serta dukungan dari Qt Designer dalam merancang tampilan antarmuka secara visual (drag-and-drop). Hal ini memisahkan antara desain UI dan logika program, sehingga pengembangan menjadi lebih rapi dan terorganisir. Integrasi antara Python dan PySide6 juga memberikan pengalaman dalam membangun aplikasi desktop yang interaktif, di mana pengguna dapat melakukan input data, melihat daftar barang, serta memahami kondisi stok melalui tampilan yang user-friendly.

B. SARAN

Sebagai pengembangan lebih lanjut, sistem manajemen inventaris gudang ini dapat ditingkatkan dengan menambahkan fitur edit data agar pengguna tidak perlu menghapus data saat terjadi kesalahan. Serta dapat ditambahkan notifikasi otomatis ketika stok berada di bawah batas minimum untuk mempercepat proses restock. Penggunaan database seperti MySQL atau SQLite juga disarankan agar data tersimpan secara permanen dan lebih aman. Antarmuka pengguna dapat diperbaiki dengan menambahkan elemen visual yang lebih informatif. Sistem juga dapat dikembangkan menjadi berbasis web atau mobile agar lebih fleksibel. Penambahan fitur login penting untuk meningkatkan keamanan. Optimalisasi algoritma diperlukan agar performa tetap baik saat data bertambah serta tak lupa dokumentasi program perlu diperjelas .

DAFTAR PUSTAKA

Grassucci, E., Barbarossa, S., & Comminiello, D. (2023). *Generative Semantic Communication: Diffusion Models Beyond Bit Recovery*. 1–14. http://arxiv.org/abs/2306.04321

Li, C., Fan, Y., Shen, Y., Wang, K., Hu, Y., Zhang, F., Pei, Y., Zheng, T., & Shi, Z. (2025). PyCinemetricsV2: Interactive computational film software based on transformers and PySide6. *SoftwareX*, *31*(August), 102299. https://doi.org/10.1016/j.softx.2025.102299

Liu, P. (2024). An in-depth study of sorting algorithms. *Applied and Computational Engineering*, *92*(1), 187–195. https://doi.org/10.54254/2755-2721/92/20241750

Nabijonovich, S. B., & Najmiddin, G. (2024). Optimizing Pyqt5 Development With Qt Designer. *Web of Teachers: Inderscience Research*, *2*(4), 254–259.

Zhang, Y. (2024). Comparative study of the execution efficiency of Python and C++Based on topological sorting. *Applied and Computational Engineering*, *34*(1), 13–17. https://doi.org/10.54254/2755-2721/34/20230288

Zhu, J. (2023). Research and Application of Qt Graphic View Framework. *Advances in Engineering Technology Research*, *4*(1), 189. https://doi.org/10.56028/aetr.4.1.189.2023

LAMPIRAN

Pembagian *jobdesk* antar anggota kelompok:

1. Dzaky Alfareza Zahari (G1A025005) : Koordinator utama tim, Inisiator Ide dan konsep awal, Menyusun laporan akhir proyek.
2. Aprilizza Auliya (G1A025007) : Merancang Desain UI di QT Designer, dan konversi ke logika Python
3. Annisa Rohiimah Sufriyani (G1A025013) : Menyusun laporan dan membantu mendesain
4. Fayola Deeba Fathinah (G1A025028) : Membantu mengumpulkan data-data yang diperlukan, dan menyusun laporan
5. Melzaqi Al-Faroh (G1A025035) : Membuat PPT dan membantu proses pengkodingan logika

Dokumentasi:

KEMENTERIAN PENDIDIKAN TINGGI, SAINS DAN TEKNOLOGI
UNIVERSITAS BENGKULU
FAKULTAS TEKNIK
PROGRAM STUDI INFORMATIKA
Jl. Wr. Supratman Kandang Limun, Bengkulu Bengkulu 38371 A
Telp: (0736) 344087, 22105 - 227

LEMBAR ASISTENSI
TUGAS BESAR PROYEK PEMROGRAMAN BERORIENTASI OBJEK

Nama Anggota Kelompok	: 1. Dzaky Alfareza Zahari	(G1A025005)
	2. Aprilizza Auliya	(G1A025007)
	3. Annisa Rohiimah Sufriyani	(G1A025013)
	4. Fayola Deeba Fathinah	(G1A025028)
	5. Melzaqi Al-Faroh	(G1A025035)
Dosen	: 1. Ir. Arie Vatresia, S.T., M.T.I., Ph.D., IPP.	
	2. Ir. Kurnia Anggriani, S.T., M.T., Ph.D.	
Asisten	: 1. Ludvie Pradya Paramitha S	(G1A024001)
	2. Aggra Kurnia Idhan	(G1A024003)
	3. Azka Fathiya	(G1A024004)
	4. Naifah Amanda Noveni	(G1A024017)
	5. Tasya Triani	(G1A024019)
	6. Atikah Putri Utami	(G1A024027)
	7. Nayla Viona Azahra	(G1A024045)
	8. Aika Kautsar Ustmut	(G1A024066)
	9. Angelita Rahmatun Annisa	(G1A024067)
	10. Ayuni Salsyabilla Putri	(G1A024096)

Laporan Tugas Besar	Catatan dan Tanda Tangan
Laporan Tugas Besar	